

Report - Version 8: LLM Parser

Introduction

Version 8 introduces the LLM Parser. Version 7 already used a local language model for routing, while argument extraction was still handled by a rule-based Generic Parser. In Version 8 the Router remains unchanged, but the Parser now uses the same local language model to extract structured arguments from the user request.

The model still runs locally inside the notebook and is downloaded from Hugging Face. The goal of the version is to replace manual argument extraction rules with a more flexible and semantic parsing step while keeping the rest of the architecture stable.

Goal of Version 8

The main goal of Version 8 is to introduce language-model-based argument parsing.

- The LLM Router continues to select the most appropriate tool.
- The LLM Parser receives the selected tool, its description, the required parameters, and the user request.
- The parser returns structured JSON-style arguments.
- The Planner, Executor, Memory Manager, and Agent remain unchanged.

What changes compared to Version 7

Version 7 introduced the LLM Router, but it still depended on a rule-based Generic Parser with helpers such as number extraction and name extraction. Version 8 removes those manual extraction functions and replaces them with an LLM Parser.

The Tool Registry becomes even more important. It now supports both LLM components: the Router uses tool names and descriptions, while the Parser uses the selected tool description and its required parameters.

New concept: LLM Parser

The LLM Parser turns argument extraction into a small language-understanding task. Instead of matching explicit patterns, the parser builds a prompt that contains the selected tool, the tool description, the required parameters, and the original request.

Selected tool:
multiplication

Tool description:
Multiply two numbers.

Required parameters:
a, b

User request:
Multiply three by four.

Return only a valid JSON object.

The local model is asked to return only JSON. The Python code then reads the model output, extracts the JSON fragment, and converts it into a Python dictionary.

Using the Tool Registry for parsing

The Tool Registry remains the shared source of truth for all tools and metadata. In Version 8, the Parser reads the selected tool description and the list of required parameters before building the prompt.

```
tool_info = tool_registry.get(tool_name)
parameters = ", ".join(tool_info["parameters"])
```

This means that parsing logic is no longer hardcoded per tool. The parser adapts its prompt based on the metadata registered in the Tool Registry.

Argument normalization

A small normalization step is kept after parsing. Its purpose is not to decide the meaning of the request, but simply to convert values such as "2" into 2 and "2.5" into 2.5 when possible.

```
def normalize_argument(value):
    if isinstance(value, str):
        if value.isdigit():
            return int(value)
        try:
            return float(value)
        except ValueError:
            return value
    return value
```

Semantic extraction is still performed by the language model. The normalization step only ensures that the Executor receives a clean data format.

Complete architecture in Version 8

The execution flow becomes:

```
Request -> LLM Router -> Planner -> LLM Parser -> Executor -> Memory Manager ->
Memory
                                     |
                                     v
                               Output
```

Agent state in Version 8

The state structure remains the same as in Version 7:

```
{
  "request": "Multiply three by four.",
  "plan": {
    "tool": "multiplication",
    "reason": "Multiply two numbers.",
    "needs_arguments": ["a", "b"]
  },
  "action": "multiplication",
  "arguments": {"a": 3, "b": 4},
  "result": 12
}
```

Tests

The tests confirm both compatibility with Version 7 and the new parsing behavior:

```
clear_memory()
tool_registry
agent("What is 2 + 3?")
agent("Hello, my name is luca.")
agent("What is the weather in Rome?")
agent("Could you calculate the product of 2 and 3?")
agent("Multiply three by four.")
get_memory()
get_last_interaction()
```

- "Could you calculate the product of 2 and 3?" verifies that the LLM Router still understands semantic intent.
- "Multiply three by four." verifies the key improvement of Version 8: the LLM Parser can extract numeric arguments expressed as words instead of digits.
- "Hello, my name is luca." verifies that name extraction is also handled by the LLM Parser.
- The weather request verifies that unsupported requests still return no tool.

What Version 8 teaches

Version 8 teaches the second practical use of a language model inside the agent architecture: semantic argument extraction. The system now uses an LLM both to decide which tool to use and to understand which values should be passed to that tool.

Limitations

- the small local model can still make parsing mistakes;
- the Planner remains rule-based and still creates a single-step plan;
- the Tool Registry remains in-memory and registration is manual;
- memory remains a simple list;
- the agent still performs one action at a time.

Conclusion

Version 8 is a natural evolution of Version 7. The Router continues to select tools semantically, and the new LLM Parser now extracts arguments semantically as well. The rest of the architecture remains deliberately stable so that the new concept is easy to isolate and understand.

The successful test with "Multiply three by four." demonstrates the key benefit of the version: manual extraction rules are no longer required to interpret arguments expressed in more natural language. The next step will be Version 9, where the project will introduce multi-step execution.